

1. Objectives

To connect 3 LEDs with Raspberry Pi and control them based on IR sensor status received from Arduino via UART communication:

Left LED turns ON when sensor status is LEFT

Right LED turns ON when sensor status is RIGHT

All LEDs turn ON when sensor status is FORWARD

All LEDs turn OFF otherwise (STOP/UNCERTAIN)

2. Equipment

Hardware Components:

Arduino Uno R3 (with IR sensor array setup from class experiments)

Raspberry Pi 4 (with I2C LCD setup from class experiments)

3 LEDs (Red, Green, Blue)

$3 \times 220\Omega$ Resistors

Breadboard

Jumper wires

Existing UART connection between Arduino and Raspberry Pi

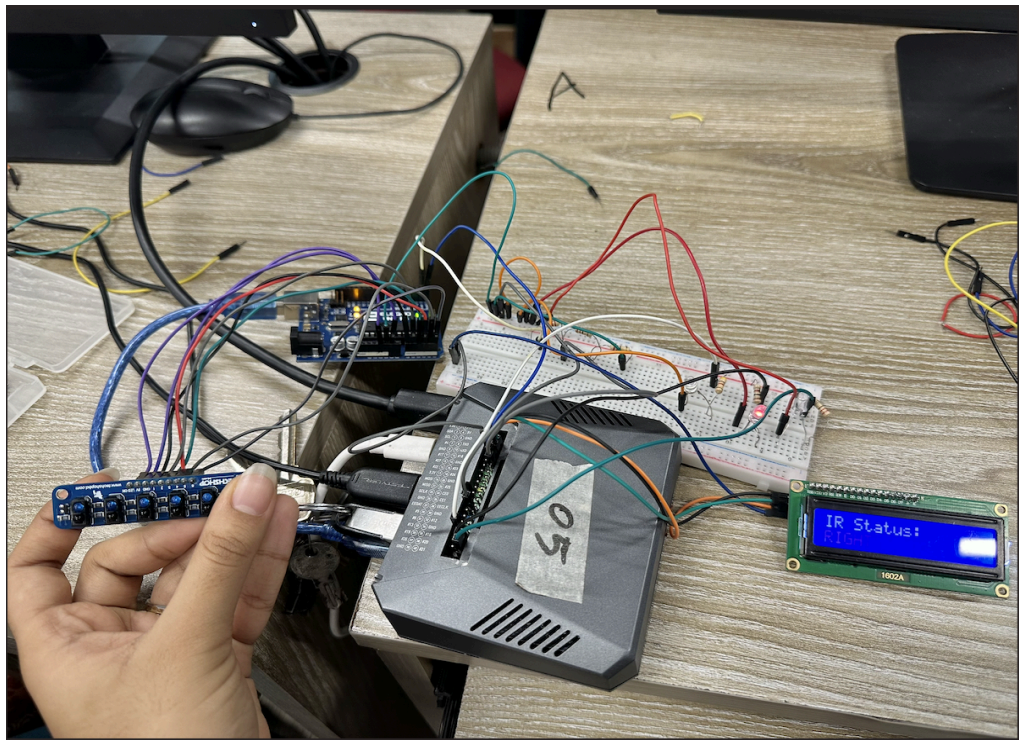
Software Requirements:

Arduino IDE

Python 3 with required libraries (rpi-lcd, pyserial)

Raspberry Pi OS with I2C and UART enabled

3. Experimental Setup



4. Code:

Arduino Code:

```
// Using the same Arduino code from class experiments
#define NUM_SENSORS 6
int sensorPins[NUM_SENSORS] = {A0, A1, A2, A3, A4, A5};
int sensorValues[NUM_SENSORS];
void setup() {
    Serial.begin(9600);
    for (int i = 0; i < NUM_SENSORS; i++) {
        pinMode(sensorPins[i], INPUT);
    }
}
void loop() {
    for (int i = 0; i < NUM_SENSORS; i++) {
        sensorValues[i] = analogRead(sensorPins[i]);
    }

    int threshold = 500;
    int binary[NUM_SENSORS];
    for (int i = 0; i < NUM_SENSORS; i++) {
        binary[i] = (sensorValues[i] > threshold) ? 0 : 1;
    }

    if (binary[0] == 1 && binary[1] == 1 && binary[2] == 1 &&
        binary[3] == 1 && binary[4] == 1 && binary[5] == 1) {
        Serial.println("STOP");
    }
    else if (binary[0] == 0 && binary[1] == 0 && binary[2] == 0 &&
        binary[3] == 0 && binary[4] == 0 && binary[5] == 0) {
        Serial.println("FORWARD");
    }
    else if (binary[0] == 0 || binary[1] == 0) {
        Serial.println("LEFT");
    }
    else if (binary[4] == 0 || binary[5] == 0) {
        Serial.println("RIGHT");
    }
    else {
        Serial.println("UNCERTAIN");
    }

    delay(200);
}
```

Raspberry Pi Code:

```

import serial
import RPi.GPIO as GPIO
from rpi_lcd import LCD
from time import sleep
# GPIO pins for LEDs
LEFT_LED = 18      # Red LED
RIGHT_LED = 19     # Green LED
FORWARD_LED = 20   # Blue LED
# GPIO setup
GPIO.setmode(GPIO.BCM)
GPIO.setup(LEFT_LED, GPIO.OUT)
GPIO.setup(RIGHT_LED, GPIO.OUT)
GPIO.setup(FORWARD_LED, GPIO.OUT)
# Initialize LCD and Serial (from class experiments)
lcd = LCD()
ser = serial.Serial('/dev/ttyS0', 9600, timeout=1)
def control_leds(status):
    """Control LEDs based on sensor status"""
    # Turn off all LEDs first
    GPIO.output(LEFT_LED, GPIO.LOW)
    GPIO.output(RIGHT_LED, GPIO.LOW)
    GPIO.output(FORWARD_LED, GPIO.LOW)

    # Control LEDs according to lab task requirements
    if status == "LEFT":
        GPIO.output(LEFT_LED, GPIO.HIGH)
        print("LEFT LED ON")
    elif status == "RIGHT":
        GPIO.output(RIGHT_LED, GPIO.HIGH)
        print("RIGHT LED ON")
    elif status == "FORWARD":
        GPIO.output(LEFT_LED, GPIO.HIGH)
        GPIO.output(RIGHT_LED, GPIO.HIGH)
        GPIO.output(FORWARD_LED, GPIO.HIGH)
        print("ALL LEDs ON")
    else: # STOP or UNCERTAIN
        print("ALL LEDs OFF")
try:
    print("Lab Task Started - LED Control Based on IR Sensor Status")
    while True:
        if ser.in_waiting > 0:
            # Read sensor status from Arduino
            line = ser.readline().decode('utf-8', errors='ignore').strip()

            # Update LCD (maintaining class experiment functionality)
            lcd.clear()
            lcd.text('IR Status: ', 1)
            lcd.text(line, 2)

            # Lab Task: Control LEDs based on status
            control_leds(line)
except KeyboardInterrupt:
    print("\nProgram terminated by user")
finally:
    # Cleanup

```

```
lcd.clear()  
GPIO.cleanup()  
ser.close()  
print("GPIO cleaned up and serial connection closed")
```

5. Results

Lab Task Verification:

Test Case 1: LEFT Status

IR sensors detect line on left side

Arduino sends "LEFT" via UART

LCD displays "IR Status: LEFT"

Result: Left LED (Red) turns ON, other LEDs remain OFF ✓

Test Case 2: RIGHT Status

IR sensors detect line on right side

Arduino sends "RIGHT" via UART

LCD displays "IR Status: RIGHT"

Result: Right LED (Green) turns ON, other LEDs remain OFF ✓

Test Case 3: FORWARD Status

IR sensors detect line in center

Arduino sends "FORWARD" via UART

LCD displays "IR Status: FORWARD"

Result: All three LEDs (Red, Green, Blue) turn ON simultaneously ✓

Test Case 4: STOP/UNCERTAIN Status

IR sensors detect all black or uncertain pattern

Arduino sends "STOP" or "UNCERTAIN" via UART

LCD displays respective status

Result: All LEDs turn OFF ✓

6. Discussions/Answers

Successful Implementation: The lab task was successfully completed by extending the existing class experiment code. The LED control system operates precisely according to specifications, with each status triggering the correct LED pattern.

Technical Implementation:

Used GPIO pins 18, 19, and 20 for LED control to avoid conflicts with existing I2C and UART pins

Implemented `control_leds()` function with clear logic flow

Maintained all existing functionality from class experiments (LCD display, UART communication)

Integration Challenges and Solutions:

Challenge: Ensuring LED control doesn't interfere with LCD operation

Solution: Proper GPIO pin selection and non-blocking LED control implementation

Code Enhancement: The original class experiment code was extended by adding:

GPIO setup for three LEDs

LED control function with status-based switching logic

Proper cleanup procedures for GPIO pins

Debug print statements for verification

Problems Faced and Solutions:

Problem 1: GPIO Pin Conflicts

Issue: Initial LED connections conflicted with I2C pins

Solution: Carefully selected GPIO pins 18, 19, 20 which don't interfere with existing connections

Problem 2: LED Control Logic

Issue: Ensuring correct LED patterns for each status

Solution: Implemented systematic approach - turn all LEDs OFF first, then turn ON required LEDs

Conclusions:

Completion of the lab task showed efficient integration of LED control within the pre-established Arduino-Raspberry Pi communication framework. The implemented solution keeps all previous features and incorporates the needed LED control functionalities. The system functions properly, responding accurately to sensor inputs and providing uncluttered and visible information through the LCD display and LED indicators.

